

BÀI GIẢI BÀI TẬP 4 - DNC

1. Mô tả việc vận chuyển dữ liệu khi thực hiện của các lệnh sau:

a. Load R5, (R20)

1. Đọc lệnh:

MAR $\leftarrow$ PC

IR $\leftarrow$ M[MAR]

2. Giải mã lệnh và đọc thanh ghi nguồn:

A $\leftarrow$ 0

B $\leftarrow$ R₂₀

PC $\leftarrow$ PC + 4

3. Thi hành lệnh:

MAR $\leftarrow$ A+B

4. Thâm nhập bộ nhớ:

MDR $\leftarrow$ M[MAR]

5. Lưu trữ kết quả:

R₅ $\leftarrow$ MDR

b. Store R7, 100(R21)

1. Đọc lệnh:

MAR $\leftarrow$ PC

IR $\leftarrow$ M[MAR]

2. Giải mã lệnh và đọc thanh ghi nguồn:

A $\leftarrow$ 100

B $\leftarrow$ R₂₁

PC $\leftarrow$ PC + 4

3. Thi hành lệnh:

MAR $\leftarrow$ A + B

MDR $\leftarrow$ R₇

4. Thâm nhập bộ nhớ:

M[MAR] $\leftarrow$ MDR

c. Add R5, R4, R20

1. Đọc lệnh:

MAR $\leftarrow$ PC

IR $\leftarrow$ M[MAR]

2. Giải mã lệnh và đọc thanh ghi nguồn:

A $\leftarrow$ R₄

B $\leftarrow$ R₂₀

PC $\leftarrow$ PC + 4

3. Thi hành lệnh:

Ngã ra của ALU $\leftarrow$ A + B

4. Thâm nhập bộ nhớ:

Bỏ qua

5. Lưu trữ kết quả:

$R5 \leftarrow \text{Ngã ra của ALU}$

d. Sub R10, R12, R24

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow R12$

$B \leftarrow R24$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

$\text{Ngã ra của ALU} \leftarrow A - B$

4. Thâm nhập bộ nhớ/nhảy:

Bỏ qua

5. Lưu trữ kết quả:

$R10 \leftarrow \text{Ngã ra của ALU}$

e. And R6, R2, R15

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow R2$

$B \leftarrow R15$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

$\text{Ngã ra của ALU} \leftarrow A \text{ and } B$

4. Thâm nhập bộ nhớ/nhảy:

Bỏ qua

5. Lưu trữ kết quả:

$R6 \leftarrow \text{Ngã ra của ALU}$

f. Or R15, R3, R7

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow R3$

$B \leftarrow R7$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

$\text{Ngã ra của ALU} \leftarrow A \text{ or } B$

4. Thâm nhập bộ nhớ:

Bỏ qua

5. Lưu trữ kết quả:

$R15 \leftarrow \text{Ngã ra của ALU}$

g. JMP R31 (Trở về chương trình chính tại địa chỉ chứa trong R31)

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow 0$

$B \leftarrow R31$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

$\text{Ngã ra của ALU} \leftarrow A + B$

4. Nhảy lần cuối:

$PC \leftarrow \text{Ngã ra của ALU}$

5. Lưu trữ kết quả (Bỏ qua)

h. BRA +5 (Nhảy không điều kiện)

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow PC$

$B \leftarrow 5 * 4$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

$\text{Ngã ra ALU} \leftarrow A + B$

4. Nhảy lần cuối:

$PC \leftarrow \text{ngã ra ALU}$

5. Lưu kết quả (không thực hiện)

e. BGT R4, +2 (Nhảy bỏ 2 lệnh nếu điều kiện trên R4 thỏa)

1. Đọc lệnh:

$MAR \leftarrow PC$

$IR \leftarrow M[MAR]$

2. Giải mã lệnh và đọc thanh ghi nguồn:

$A \leftarrow PC$

$B \leftarrow 2 * 4$

$PC \leftarrow PC + 4$

3. Thi hành lệnh:

Xét điều kiện trên R4,

Nếu thỏa điều kiện

$\text{Ngã ra ALU} \leftarrow A + B$

4. Nhảy lần cuối:

PC $\leftarrow$ ngĩa ra ALU
Nếu không thỏa điều kiện
Thực hiện lệnh kế tiếp

2.

a) **Tính $T_{\text{pipelined}}$:**

Ta có thể xem trong kỹ thuật ống dẫn cơ bản CPU thực hiện đồng thời 5 lệnh trong thời gian $T_{\text{unpipelined}} + 5 \cdot T_L$, với 5 là số giai đoạn của một lệnh, T_L là thời gian chốt ống dẫn (Pipeline latch Latency). Ta có thời gian **tương đương** để thực hiện 1 lệnh trong kỹ thuật ống dẫn cơ bản là:

$$T_{\text{pipeline}} = \frac{T_{\text{unpipeline}}}{5} + T_L$$
$$T_{\text{pipeline}} = \frac{25\text{ns}}{5} + 1\text{ns} = 6\text{ns}$$

Tính tốc độ xử lý (Throughput) ký hiệu R:

Mỗi lệnh được thực hiện trong thời gian $T_{\text{pipelined}}$. Số lệnh thực hiện trong 1 s là:

$$R = \frac{1}{T_{\text{pipeline}}} = \frac{1}{6 \cdot 10^{-9}} = 166.7 \text{ triệu lệnh/s}$$

Nhận xét:

Tốc độ xử lý khi không dùng KT ống dẫn

$$R_0 = (1/T_{\text{unpipeline}}) = 1/(25 \cdot 10^{-9}) = 40 \text{ triệu lệnh/s}$$

Tốc độ xử lý khi dùng KT ống dẫn **theo lý thuyết** là:

$$R = R_0 \cdot 5 = 40 \cdot 5 \text{ triệu lệnh} = \mathbf{200 \text{ triệu lệnh/s.}}$$

Kết luận: Trong thực tế, tốc độ xử lý nhỏ hơn lý thuyết, vì cần có thời gian chốt ống dẫn.

b) **Trong kỹ thuật siêu ống dẫn bậc $n = 10$** , mỗi giai đoạn cơ bản chia thành 10 giai đoạn con. Tổng số giai đoạn tương ứng với 1 lệnh là: $m = 5 \cdot 10$ giai đoạn. Ta có thể xem trong kỹ thuật siêu ống dẫn bậc $n=10$, CPU thực hiện đồng thời 50 lệnh trong thời gian $T_{\text{unpipelined}} + 5 \cdot 10 \cdot T_L$, vì vậy mỗi lệnh được thực hiện trong thời gian $T_{\text{pipelined}}$ (thời gian **tương đương** để thực hiện 1 lệnh) là:

$$T_{\text{pipeline}} = \frac{25\text{ns}}{5 \cdot 10} + 1\text{ns} = 1.5\text{ns}$$

Tốc độ xử lý: Số lệnh thực hiện trong 1 s là:

$$R = \frac{1}{T_{\text{pipeline}}} = \frac{1}{1.5 \cdot 10^{-9}} = 666.7 \text{ triệu lệnh/s.}$$

Nhận xét: Theo lý thuyết, R dùng KT siêu ống dẫn bậc 10 là $R_0 \cdot 5 \cdot 10 = 200 \text{ triệu} \cdot 10 = 2000 \text{ triệu lệnh}$.

Vậy tốc độ trong thực tế nhỏ tốc xử lý trong lý thuyết. Lý do là phải có thời gian chốt ống dẫn cho mỗi gian đoạn con. Thời gian chốt ống dẫn đã tăng n lần.

c) Trường hợp thời gian thực hiện các giai đoạn là khác nhau, cụ thể là 5, 7, 3, 6 và 4 ns lần lượt cho các giai đoạn. Trong trường hợp này, chu kỳ xung nhịp $T_{\text{pipelined}}$ phải bằng thời gian của giai đoạn dài nhất cộng với thời gian chốt ống dẫn. Ta có:

$$T_{\text{pipelined}} = 7 + 1 = 8 \text{ ns}$$

$$R = 1 / T_{\text{pipelined}} = 1 / (8 \cdot 10^{-9}) = 125 \text{ triệu lệnh/s}$$

3. Hãy hoàn thành bảng trạng thái sau đây của phép toán sau SUBCC Ri,Rj (phép trừ, lưu các bit trạng thái vào thanh ghi trạng thái):

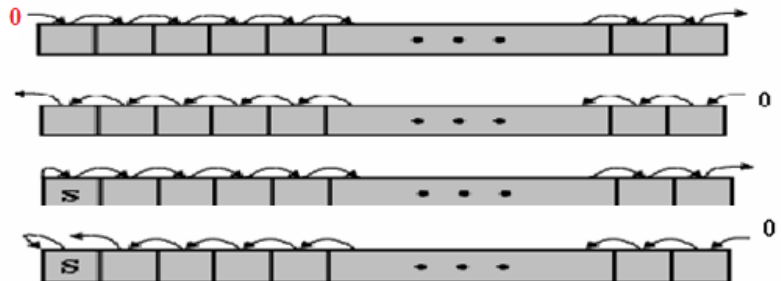
Kết quả	S	Z
=	X	1
>	0	0
<	1	X
>=	0	X
<=	1	0
	0	1

Trong các trường hợp trên, S và Z có thể lấy giá trị là 0 hoặc 1, trường hợp không cần xét bit trạng thái đó thì ghi dấu x.

4. Trên một hệ thống với từ dữ liệu 8 bit, giả sử số liệu trên thanh ghi A là N, cho biết kết quả:

Định nghĩa:

- Dịch phải logic
- Dịch trái logic
- Dịch phải số học
- Dịch trái số học



Ví dụ:

Trên một hệ thống với từ dữ liệu 8 bit, giả sử số liệu trên thanh ghi A là N, cho biết kết quả:

- Lệnh dịch trái logic 2 bit trên thanh ghi A, với $N = 25$. Nhận xét.
- Lệnh dịch phải logic 2 bit trên thanh ghi A, với $N = 25$. Nhận xét.
- Lệnh dịch trái số học 3 bit trên thanh ghi A, với $N = -15$. Nhận xét.

- d. Lệnh dịch trái số học 3 bit trên thanh ghi A, với $N = -17$. Nhận xét.
e. Lệnh dịch phải số học 3 bit trên thanh ghi A, với $N = 15$. Nhận xét.
f. Lệnh dịch phải số học 2 bit trên thanh ghi A, với $N = 16$. Nhận xét.
g. Lệnh dịch trái số học 2 bit trên thanh ghi A, với $N = 16$. Nhận xét.

Giải:

a. Dịch trái logic: $N=25, m=2$

Trước khi dịch 25
Sau khi dịch 100

0	0	0	1	1	0	0	1
0	1	1	0	0	1	0	0

Nhận xét:

$100 = 25 \times 2^2 \rightarrow$ phép **dịch trái logic** 2 bit một số dương (không có dấu) tương đương với phép nhân cho 2^2 .

Tổng quát, phép dịch trái logic m bit một số dương (không có dấu) N tương đương với phép nhân N cho 2^m .

Tuy nhiên, một số không có dấu n bit chỉ biểu diễn được con số dương lớn nhất là $(2^n - 1)$. Vì vậy, kết luận trên chỉ đúng thì $N \times 2^m \leq (2^n - 1)$, nghĩa là tùy thuộc vào N, mà chọn số bit dịch m thích hợp. Trong bài tập này $N = 25$, nên m lớn nhất là 3.

b. Dịch phải logic: $N=25, m=2$

➤ Trường hợp theo định nghĩa phép dịch phải logic như sau (bit 1 nhảy vào)::



Trước khi dịch 25
Sau khi dịch 6 (số không có dấu)

0	0	0	1	1	0	0	1
0	0	0	0	0	1	1	0

Nhận xét:

$6 = 25 / 4 \text{ dư } 1 \rightarrow 6 = 25 / 2^2 \rightarrow 1 \rightarrow 6 = 25 \times 2^{-2} \rightarrow 1$ Phép dịch phải logic 2 bit một số dương (số không có dấu) tương đương với phép chia cho 2^2 (nhân cho 2^{-2}) lấy phần nguyên.

Phép dịch phải logic m bit một số dương (số không có dấu) tương đương với phép chia cho 2^m (nhân cho 2^{-m}).

➤ Trường hợp theo định nghĩa phép dịch phải logic như sau (bit 0 nhảy vào):



Trước khi dịch 25
Sau khi dịch 6 (số không có dấu)

0	0	0	1	1	0	0	1
0	0	0	0	0	1	1	0

Nhận xét:

Phép dịch phải logic m bit một số dương (số không có dấu) tương đương với phép chia cho 2^m (hay nhân cho 2^{-m}) lấy phần nguyên.

Qui định bit 0 hay bit 1 nhảy vào là tùy thuộc vào CPU, **khi thì ta chọn trường hợp bit 0 nhảy vào.**

c. Dịch trái số học: $N = -15, m = 3$

• Dịch trái số học



Trước khi dịch	-15 (số bù 2)	1	1	1	1	0	0	0	1
Sau khi dịch	-120 (số bù 2)	1	0	0	0	1	0	0	0

Nhận xét:

$-120 = -15 \times 2^3 \rightarrow$ phép **dịch trái đại số** số 3 bit một **số âm** (số có dấu kiểu bù 2) tương đương với phép nhân cho 2^3 .

Tổng quát, phép dịch trái **đại số** m bit một số **số âm** (số có dấu kiểu bù 2) N tương đương với phép nhân N cho 2^m .

Tuy nhiên, một số có dấu n bit chỉ biểu diễn được con số âm nhất là:

$-(2^{n-1}-1)$. Vì vậy, kết luận trên chỉ đúng thì $N \times 2^m \geq -(2^{n-1}-1)$, nghĩa là tùy thuộc vào N, mà chọn số bit dịch m thích hợp. Trong bài tập này $N = -15, n = 8$, số âm nhất là -127, nên m lớn nhất là 3.

Kết luận:

- Dịch logic dùng để thực hiện phép nhân (dịch trái)/chia (dịch phải, trường hợp bit 0 nhảy vào) một số nguyên dương (không có dấu) cho lũy thừa của 2.
- Dịch số học dùng để thực hiện phép nhân (dịch trái) hoặc chia (dịch phải, trường hợp bit 0 nhảy vào) một số đại số (có dấu) cho lũy thừa của 2.

d. Dịch phải số học: $N = -17, m = 3$.

• Dịch phải số học



Trước khi dịch	-17 (số bù 2)	1	1	1	0	1	1	1	1
Sau khi dịch	-3 (số bù 2)	1	1	1	1	1	1	0	1

Giải thích: $1110\ 1111 \rightarrow 11110\ 111 \rightarrow 111110\ 11 \rightarrow 1111110\ 1$

Bù 1 $\rightarrow 0000\ 0010$ Bù 2 $\rightarrow 0000\ 0011 = 3 \rightarrow$ KQ là -3

$-2 = -16/8 = -16/(2^3) = -18 \times 2^{-3} \rightarrow -(17-1) \times 2^{-3}$

Nhận xét:

- không có quan hệ với phép nhân/chia 2^m .

e. Dịch phải số học: $N = -15, m = 3$

Trước khi dịch	-15 (số bù 2)	1	1	1	1	0	0	0	1
Sau khi dịch	-2 (số bù 2)	1	1	1	1	1	1	1	0

Nhận xét:

Phép dịch phải đại số n bit một số âm **không** có quan hệ với phép nhân cho 2^{-n} (phép chia cho 2^n). Ta thấy không tương đương với phép chia cho 2^n là vì không chia chắn

f. Dịch phải số học: $N = 16, m = 2$

Trước khi dịch	16 (số dương)	0	0	0	1	0	0	0	0
Sau khi dịch	4 (số dương)	0	0	0	0	0	1	0	0

Nhận xét:

$4 = 16 \times 2^{-2} \rightarrow$ phép **dịch phải đại** số 2 bit một **số dương** tương đương với phép nhân cho 2^{-2} (chia cho 2^2).

h. Dịch trái số học: $N=16, m=2$

Trước khi dịch	16 (số dương)	0	0	0	1	0	0	0	0
Sau khi dịch	64 (số dương)	0	1	0	0	0	0	0	0

Nhận xét:

$64 = 16 \times 2^2 \rightarrow$ phép **dịch trái** số học 2 bit một **số dương** tương đương với **phép nhân** cho 2^2 .

Tổng quát, phép **dịch phải số học** m bit một **số dương** N tương đương với **phép chia** N cho 2^m **lấy phần nguyên**. Tuy nhiên, m phải thỏa điều kiện: $2^m \leq N$

Phép **dịch trái số học** m bit một **số dương** N tương đương với nhân N cho 2^m . Tuy nhiên, một số có dấu n bit chỉ biểu diễn được con số dương lớn nhất là $(2^{n-1} - 1)$. Vì vậy, kết luận trên chỉ đúng thì $N \times 2^m \leq (2^{n-1} - 1)$, nghĩa là tùy thuộc vào N , mà chọn số bit dịch m thích hợp. Trong bài tập này $N = 16$, nên m lớn nhất là 2.

- Kiến trúc RISC dùng thanh ghi R31 để lưu địa chỉ trở về và thanh ghi R30 làm con trỏ ngăn xếp. Xét trường hợp từ chương trình chính gọi thủ tục P1, rồi từ thủ tục P1 gọi thủ tục P2 một lần, sau đó trở về thủ tục P1, rồi trở về chương trình chính. Cho biết lệnh JMPL Ri là lệnh gọi thủ tục có địa chỉ đầu tiên nằm trong Ri, lệnh JMP R31 là lệnh kết thúc thủ tục trở về chương trình gọi thủ tục. Giả sử thủ tục P1 bắt đầu từ địa chỉ ghi trong R1 và thủ tục P2 bắt đầu từ địa chỉ ghi trong R2. Hãy mô tả chương trình thực thi các tác vụ này.**

Giải:

JMPL R1 // Gồm các thao tác: $R31 := PC$; $PC := R1$ //

Gọi thủ tục P2:

ADDI R30, R30, 4

STORE R31, (R30)

JMPL R2 // Gồm các thao tác: $R31 := PC$; $PC := R2$ //

Trở về thủ tục P1:

JMP R31

Trở về chương trình chính:

LOAD R31, (R30)

SUBI R30, R30, 4

JMP R31